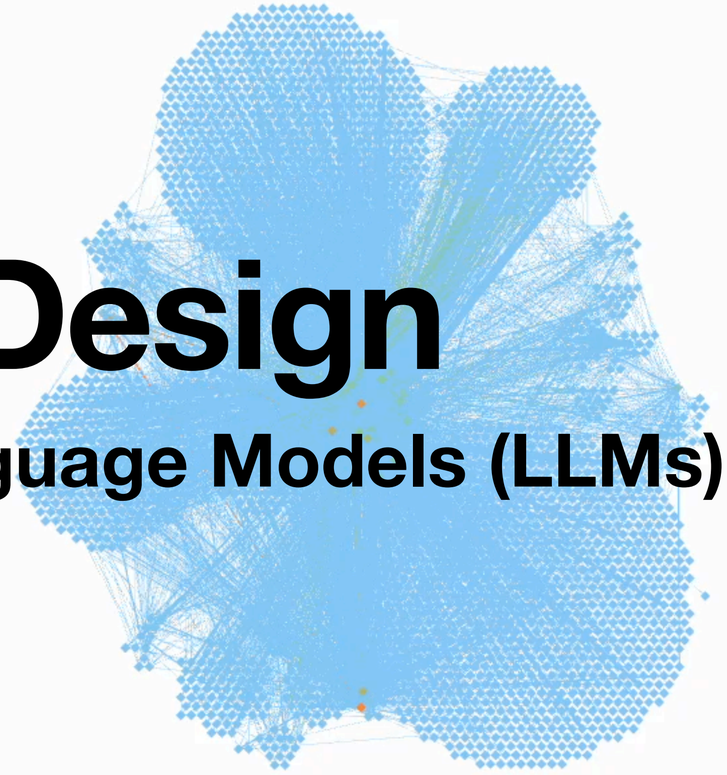
ISA104
AI Application Design

L2: Introduction to Large-Language Models (LLMs)



Dr. Dapeng Dong

“I think the brain is essentially a computer and consciousness is like a computer program. It will cease to run when the computer is turned off. Theoretically, it could be re-created on a neural network, but that would be very difficult, as it would require all one’s memories.”

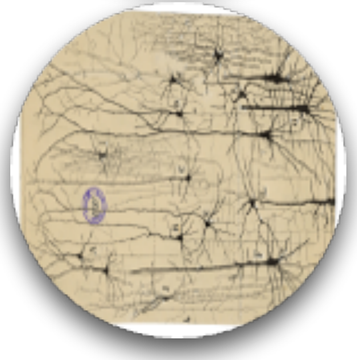
— *Stephen Hawking*

Session Objectives

- Provide an overview of the history and evolution of language models, highlighting key milestones and developments.
- Define and explain large language models, including their variations such as types and architectures.
- Describe the components of large language models, explaining how they work together to process and generate text.
- Exploring the self-attention mechanism and its role in capturing complex contextual relationships.
- Discuss the potential applications of foundation models.

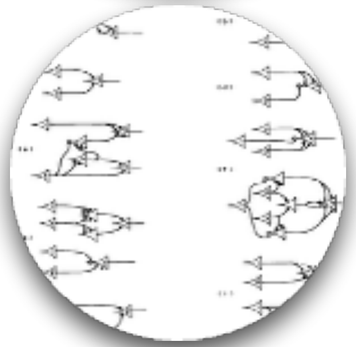
A Brief History of Large Language Models (1)





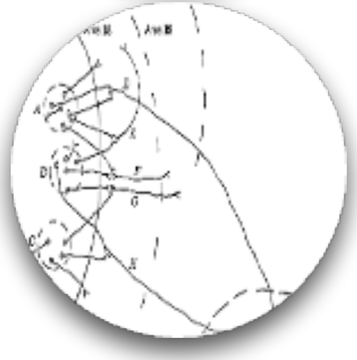
1888 Human Brain Neuron Visualisation
Santiago Ramon y Cajal was awarded the Nobel Prize in Physiology in 1906 for his groundbreaking work in revealing the structure of natural neural networks.





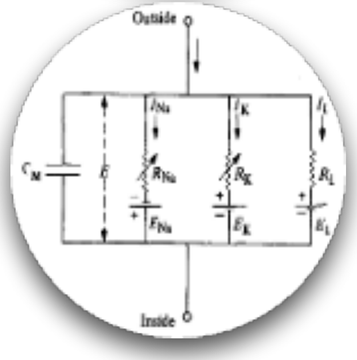
1943 Mathematical Model of Neuron
Neurophysiologist **Warren McCulloch** & mathematician **Walter Pitts** developed the first mathematical model of neuron in their work: "*A logical calculus of the ideas immanent in nervous activity*"





1949 Hebbian Learning
Physiological psychologist **Donald Hebb** introduced the fundamental model for learning and adjusting synaptic weights between neurons.

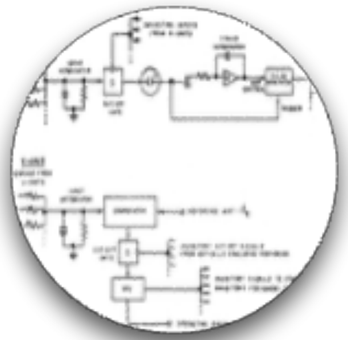




1952 Neuron Transporter Models
Biophysicists **Alan Hodgkin** and **Andrew Huxley** were awarded the 1963 Nobel Prize in Physiology for their groundbreaking research on the physiological mechanisms underlying neural excitation.

A Brief History of Large Language Models (2)





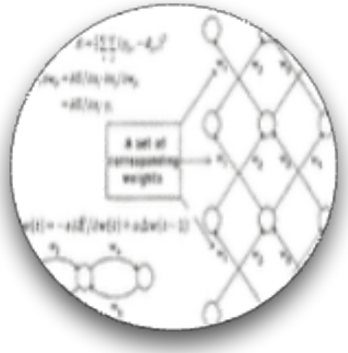
1958 The Perceptron
Frank Rosenblatt and his team developed a software system and a hardware implementation — the Mark I Perceptron. The device, introduced in 1958, was capable of adaptively adjusting its neural weights.





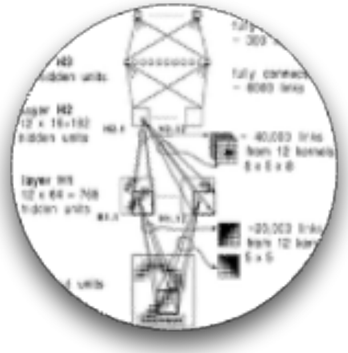
1982 Recurrent Neural Networks
John Hopfield is perhaps best known for his work on the Hopfield Network, which is a type of recurrent neural network that can store and recall patterns in a stable manner, published in 1982





1986 Backpropagation & MLP
David Rumelhart is a renowned American computer scientist who made significant contributions to the field of AI. He is best known for his work on Backpropagation and Multilayer Perceptrons (MLPs).

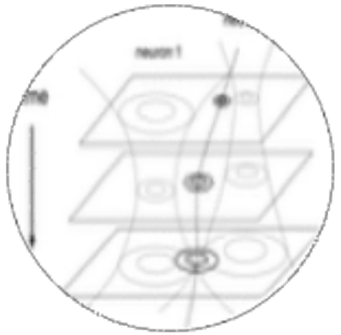




1989 Convolutional Neural Networks (CNN)
Yann LeCun is a French computer scientist. His work, such as CNN, has had a profound impact on the development of AI and deep learning technologies.

A Brief History of Large Language Models (3)



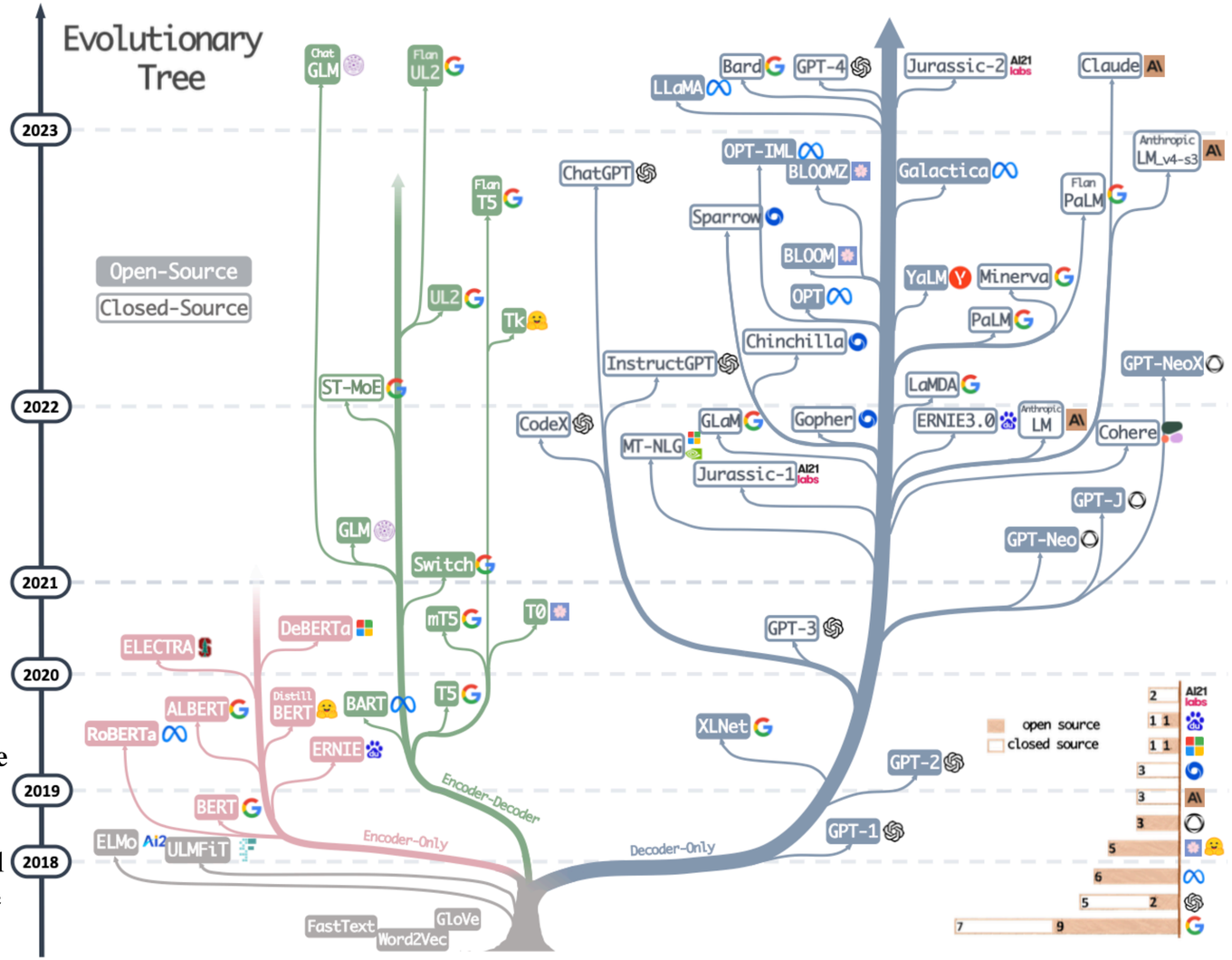


2006 Deep Belief Networks
Geoffrey E. Hinton is a Canadian computer scientist, who was awarded the 2024 Nobel Prized in Physics. His work in Boltzmann Machines & deep belief networks has great influential figures in AI and deep learning.





2017 The Transformer Architecture
The Transformer architecture, published in 2017, has been a game-changer in the field of AI and Natural Language Processing, enabling more accurate and efficient processing of human language.



Source: Yang, Jingfeng, et al. 'Harnessing the Power of LLMs in Practice: A Survey on ChatGPT and Beyond'. ACM Transactions on Knowledge Discovery from Data, vol. 18, no. 6, Jul. 2024, pp. 1–32. DOI.org (Crossref), https://doi.org/10.1145/3649506.

What is an LLM?

*"**Large language models**, also known as **LLMs**, are very large deep learning models that are pre-trained on vast amounts of data."*

— *Amazon.com*

*"**Large language models (LLMs)** are a category of foundation models trained on immense amounts of data making them capable of understanding and generating natural language and other types of content to perform a wide range of tasks."*

— *IBM.com*

*"**Large language models (LLMs)** are advanced AI systems that understand and generate natural language, or human-like text, using the data they've been trained on through* machine learning *techniques."*

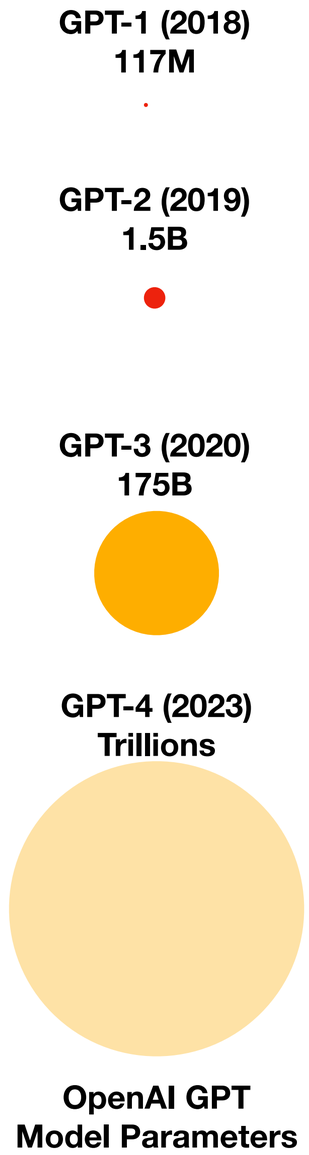
— *Microsoft.com*

*"A **large language model (LLM)** is an artificial intelligence system that has been trained on a vast dataset, often consisting of billions of words taken from books, the web, and other sources, to generate human-like, contextually relevant responses to queries."*

— *Oracle.com*

*"**LLMs** are a specially designed subset of machine learning known as deep learning, which uses algorithms trained on large data sets to recognize complex patterns."*

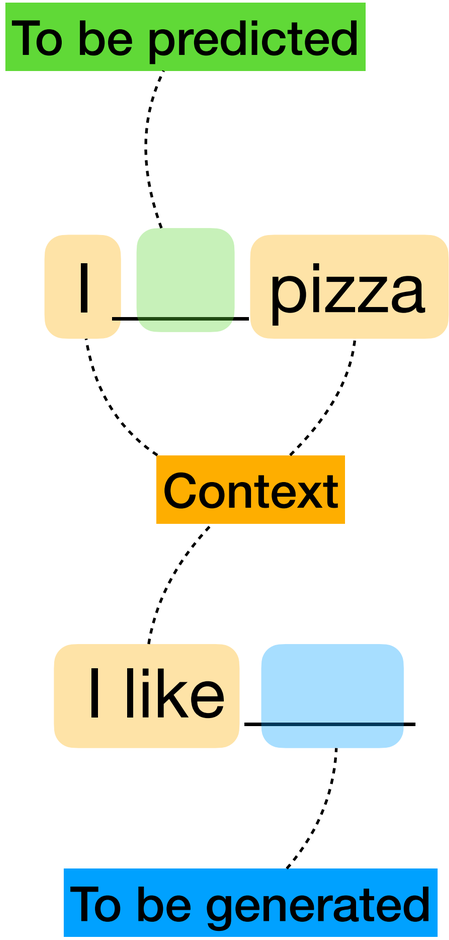
— *SAP.com*



Types of Language Models

There are two main types of language models:

- **Masked language model**
  - Trained to predict missing words in a sentence by leveraging context from both before and after the missing word.
  - Commonly used for tasks such as sentiment analysis, classification, and code debugging, where understanding the overall context is important.
  - Primarily using the Encoder submodule of a transformer architecture
- **Autoregressive language model**
  - Trained to predict the next word in a sentence using only the previous words as context.
  - Commonly used for generating text one token at a time, such as creative writing
  - Primarily based on the Decoder submodule of a transformer architecture



Foundation Models vs. Fine-tuned Models

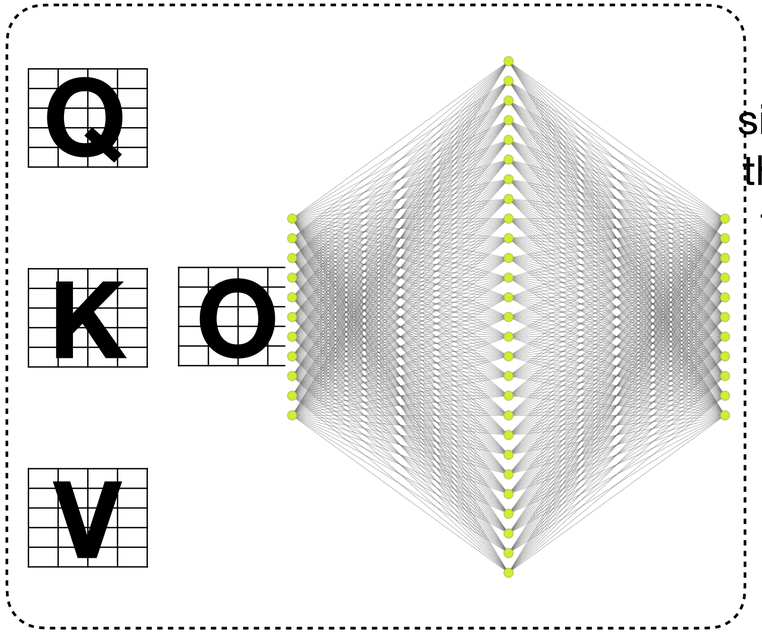
Foundation Models provide a general-purpose foundation for understanding language or other tasks, while Fine-Tuned Models focus on optimizing performance for specific applications and domains.



Common Crawl
StackExchange Q&A
GitHub Code
Reddit
Semantic Scholar
Project Gutenberg
Wikipedia
RedPajama
Pile
……

Unlabelled Data
(Billion — Quadrillion Tokens)

Pre-train

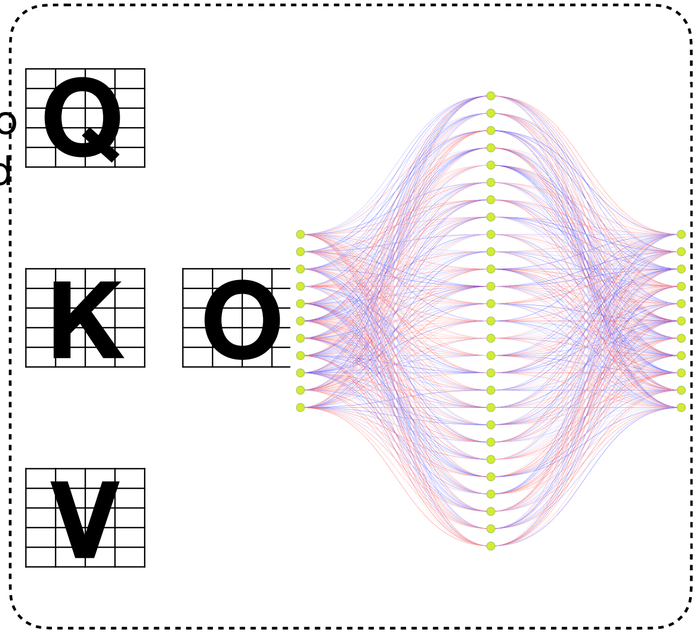


Pre-trained model
Also known as: Foundation or Base Model, capable of performing text generation

Labelled Data
(Much smaller size compared to the dataset used for pre-training

Fine-tune





Fine-tuned model

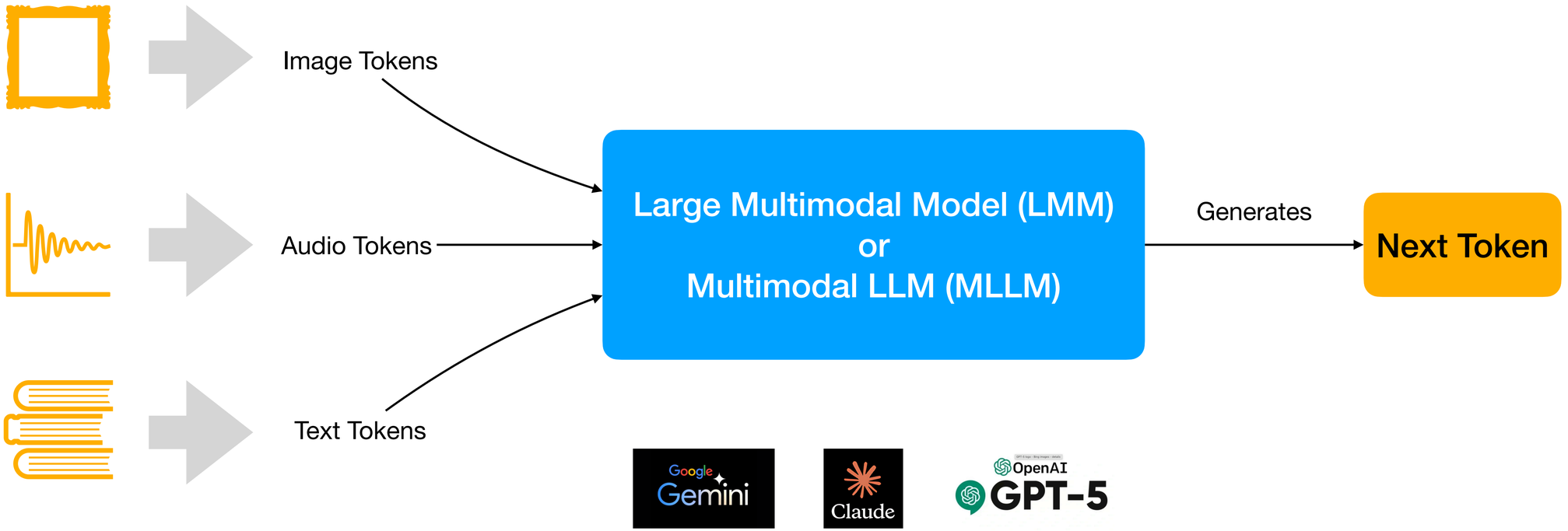
- Classification
- Translation
- Calculation
- Mathematics
- Personal assistant

……

LLM vs. LMM

*"**Multimodal AI** refers to machine learning models capable of processing and integrating information from multiple modalities or types of data. These modalities can include text, images, audio, video and other forms of sensory input."*

— IBM.com



Data modalities are distinct formats or structures of input data, such as text, images, audio, video, and time-series data, which necessitate unique approaches for labeling and analysis.

ChatGPT vs. InstructGPT

ChatGPT

- Engineered specifically for conversational applications.
- ChatGPT can maintain context over multiple conversations.
- Be able to produce human-like dialogue and engage in conversations.
- Ideal for Chatbots, interactive applications, or customer support alike applications.

InstructGPT

- Optimised for following explicit instructions and generate outputs that closely align with user commands.
- InstructGPT focuses on understanding and executing directives rather than maintaining a conversational flow.
- Be able to generate formal, precise responses.
- Ideal for content rewriting, code generation, data summarisation, and follow instructions to complete tasks with accuracy

Measuring the Size of Training Data

*"The number of tokens in a model's training dataset isn't the same as its number of **training tokens**."*

— Huyen, Chip. AI Engineering: Building Applications with Foundation Models. First edition, O'Reilly, 2025.

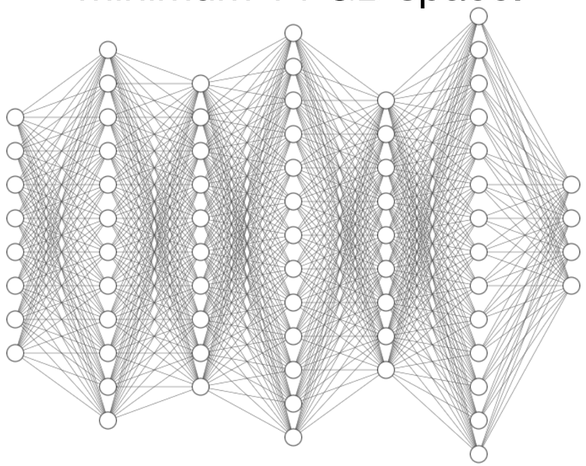
- The size of training dataset is commonly measured in terms of the number of tokens.
  - Note that depending on the tokenization methods used by an LLM, the same dataset can result in a different number of tokens.
- The number of **training tokens** measures the total number of tokens that the model is trained on.
  - E.g., if a dataset contains 1 trillion tokens, and a model is trained on that dataset for two epochs, the number of training tokens will be 2 trillions.

*"A complete pass through the whole training set is known as a training **epoch**."*

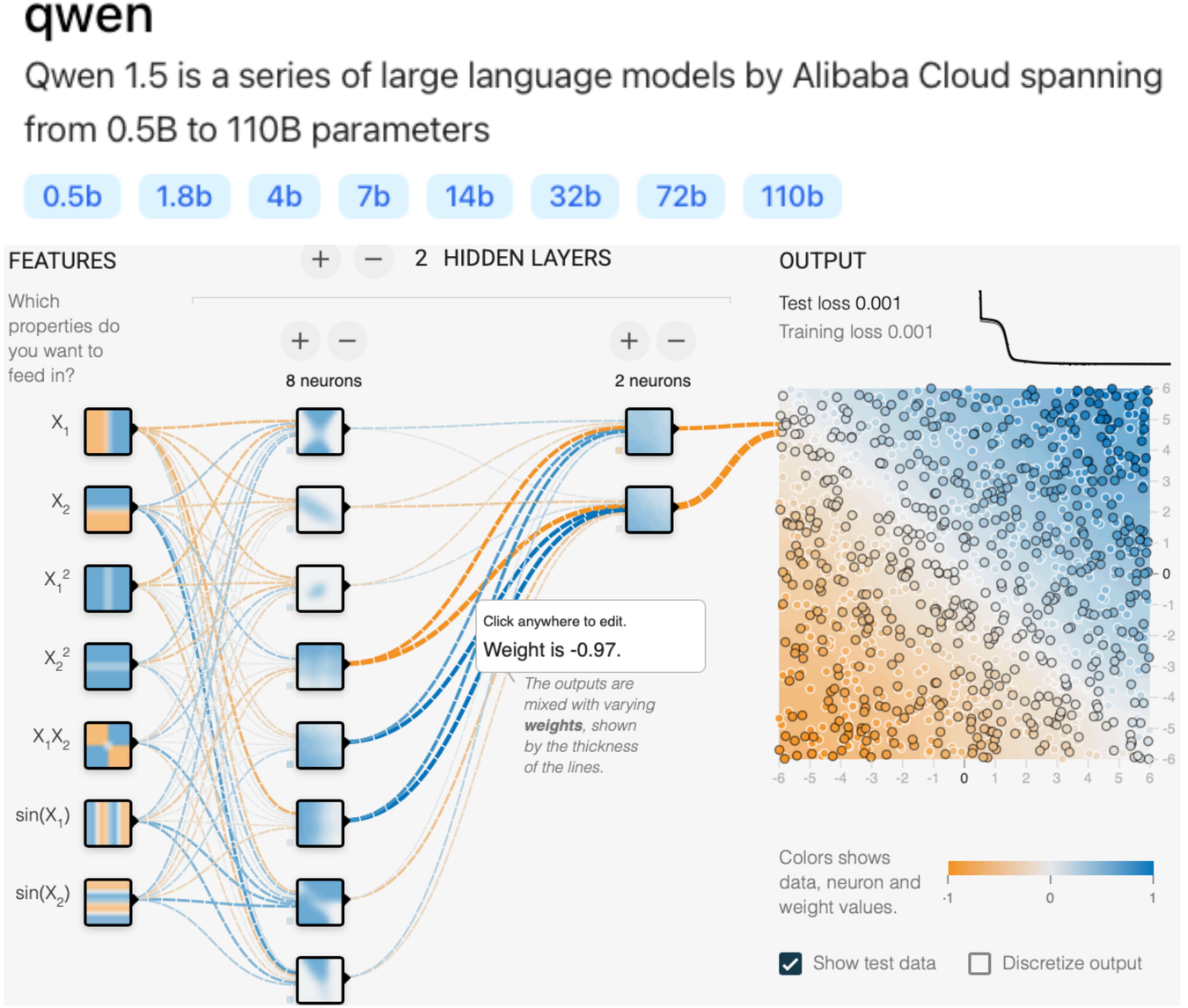
— Bishop, Christopher M., and Hugh Bishop. Deep Learning: Foundations and Concepts. Springer, 2024.

Measuring the Size of a Model

- The most common and standard way to measure the size of a large language model is by counting its total number of **trainable parameters**.
  - E.g., Qwen 7b model contains approximately 7 billion *trainable parameters* or *weights*.
- Technically, each weight is represented by a number, which is typically stored using 32 bits (4 bytes, float) or 16 bits (2 bytes, half-precision float), e.g., 0.6 or -0.97.
  - If we use 2 bytes to store a floating-point number in the computer, a model consisting of 7 billion trainable parameters will theoretically take minimum 14 GB space.



The connections between nodes in the network are trainable (adjustable) parameters or weights.



Reference: https://playground.tensorflow.org

Measuring Model Training Compute Requirements



- Most of the computational tasks within a model's training process involve matrix operations on floating-point numbers.
- A common unit for measuring a model's compute requirement is **FLOP** (floating-point operation), e.g.:
  - A single NVIDIA H100 NVL Datacenter GPU can deliver a maximum of 60 TFLOP/s (60 tera floating-point operations per second, i.e., $6 * 10^{13}$ FLOPs per second or $60 * 60 * 24 * 6 * 10^{13} = 5.251392 * 10^{18}$ FLOPs per day);
  - A single NVIDIA RTX 8000 consumer-level GPU can deliver a maximum of 16.3 TFLOP/s or $1.40832 * 10^{18}$ TFLOPs/day.
- GPT-3 (175B) was trained using $3.14 * 10^{23}$ FLOPs
  - Training GPT-3 (175B) on a single H100 NVL GPU would take more than 160 years!
  - Training GPT-3 (175B) on a single RTX 8000 GPU would take more than 600 years!

SPECIFICATIONS	
GPU Memory	**48 GB GDDR6**
Memory Interface	**384-bit**
Memory Bandwidth	**672 GB/s**
ECC	**Yes**
NVIDIA CUDA Cores	**4,608**
NVIDIA Tensor Cores	**576**
NVIDIA RT Cores	**72**
Single-Precision Performance	**16.3 TFLOPS**
Tensor Performance	**130.5 TFLOPS**
NVIDIA NVLink	**Connects 2 Quadro RTX 8000 GPUs[1]**
NVIDIA NVLink bandwidth	**100 GB/s (bidirectional)**
System Interface	**PCI Express 3.0 x 16**
Power Consumption	**Total board power: 295 W Total graphics power: 260 W**
Thermal Solution	**Active**
Form Factor	**4.4" H x 10.5" L, Dual Slot, Full Height**
Display Connectors	**4xDP 1.4, VirtualLink (1)**
Max Simultaneous Displays	**4x 3840 x 2160 @ 120 Hz, 4x 5120x2880 @ 60 Hz, 2x 7680x4320 @ 60 Hz**
Encode / Decode Engines	**1X Encode, 1X Decode**
VR Ready	**Yes**
Graphics APIs	**DirectX 12.0[7], Shader Model 5.1[7], OpenGL 4.6[8], Vulkan 1.1[8]**
Compute APIs	**CUDA, DirectCompute, OpenCL™**

References:

https://lambdalabs.com/

Brown, Tom B., et al. 'Language Models Are Few-Shot Learners'. arXiv:2005.14165, arXiv, 22 Jul. 2020. arXiv.org, https://doi.org/10.48550/arXiv.2005.14165.

Open Research Questions

We can provide data to a Large Language Model for it to learn from, but how can we make it effectively forget a specific piece of information?

Let's say we're training a language model on a dataset about historical events, and we want to prevent it from remembering that Napoleon Bonaparte was exiled twice. We provide the model with a lot of data about Napoleon, but we also include some "forgetting" cues in the form of additional training examples that contradict or downplay this information.

For instance:

The original dataset may stated that: *Napoleon Bonaparte was exiled to Elba in 1814.*

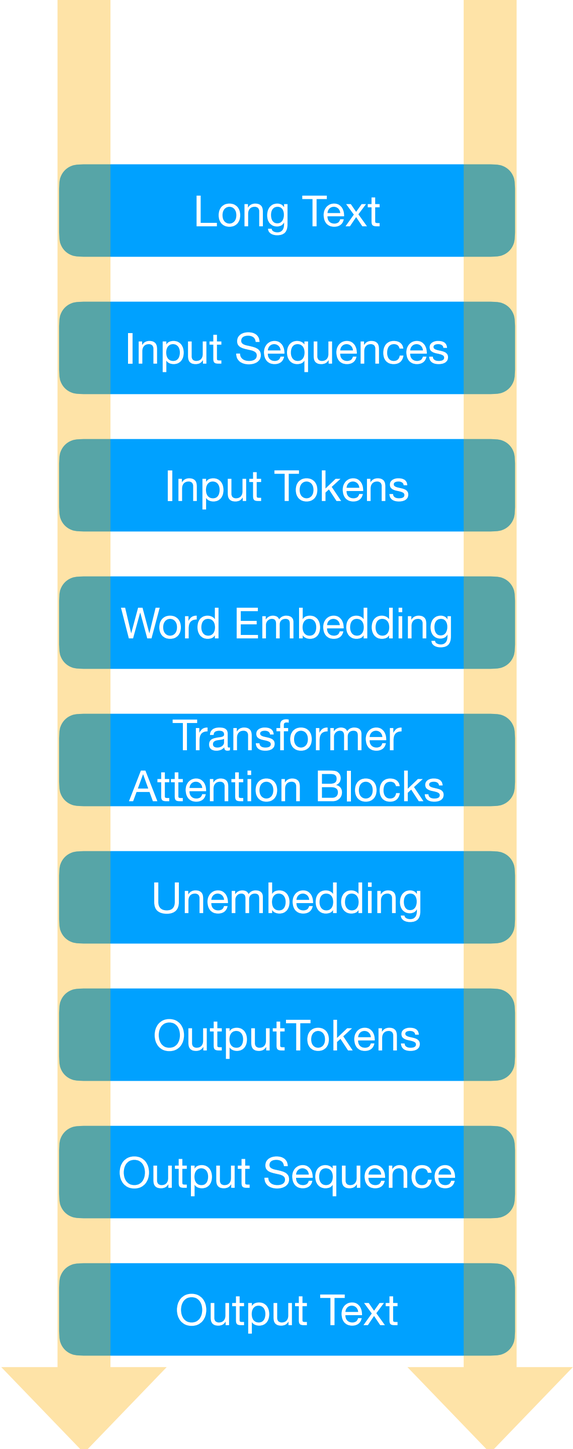
In the additional training examples, we may include that: *Some historians argue that Napoleon's exile to Elba was a temporary setback, and he maintained significant influence from abroad.*

By presenting the model with these contradictory examples, we could help it learn to be uncertain or forgetful about this specific piece of information, but the effectiveness of this approach is unclear; are there any other effective approaches?

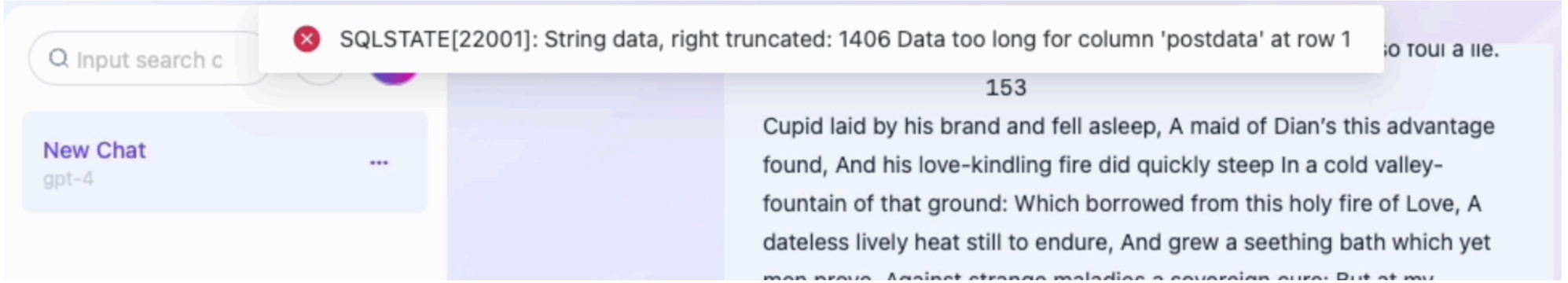
How LLMs Work

General Processes:

1. To process a long document or text, it's first broken down into multiple sequences of text
2. Each sequence is then converted into tokens
3. These input tokens are mapped to word embeddings
4. The embedding vectors then pass through the Transformer layers, allowing the model to capture complex relationships between words and generate contextualized representations
5. The unembedding mechanism is used to retrieve tokens from their embedded representations
6. From the list of possible tokens, a simple response is sampled using techniques such as probability-based sampling or beam search
7. The decoded tokens are then transformed back into text that can be understood by humans



The Prompt Length Limitation



Long Text

Input Sequences

When we input the entire of Shakespeare's "The Sonnets", which is approximately 56KB, to GPT-4, an error message appeared.

- The self-attention mechanisms in Transformer architectures demand significant computational resources.
- There are memory limitations in production environments, such as cloud computing platforms.

A well-calibrated prompt length is crucial for achieving a successful AI-powered workflow.

(2025)

Model	Max. Token Limit	Max. Word Limit
GPT-4 Standard	8K	6K - 6.5K
GPT-4 Extended	32K	24K - 26K
GPT-4 Turbo	128K	96K - 100K

Source: https://www.byteplus.com/en/topic/546386?title=how-long-can-a-gpt-4-prompt-be-2025-guide

Dealing with Long Inputs

General guidelines

- Splitting a long input into chunks based on the model's suggested length, e.g., 15K character limit for GPT (2023 version).
- Using appropriate prompts to instruct GPT to recall previously supplied chunks.
- Instructing GPT to process all chunks and generate a response once all chunks have been received.

An Example Prompt for Chunked Inputs:

The total length of the content that I want to send you is too large to send in only one piece.
For sending you that content, I will follow this rule:
[START PART 1/10]
this is the content of the part 1 out of 10 in total
[END PART 1/10]
Then you just answer: "Received part 1/10"
And when I tell you "ALL PARTS SENT", then you can continue processing the data and answering my requests.

Tokenization

88 Characters; 17 Words; 19 Tokens

Willingness	W 54
to	illing 9585
learn	ness 2136
is	to 311
important	learn 4048
,	is 374
but	important 3062
willingness	, 11
to	but 719
act	willingness 39375
on	to 311
what	act 1180
you	on 389
learn	what 1148
is	you 499
critical	learn 4048
.	is 374
	critical 9200
	. 13

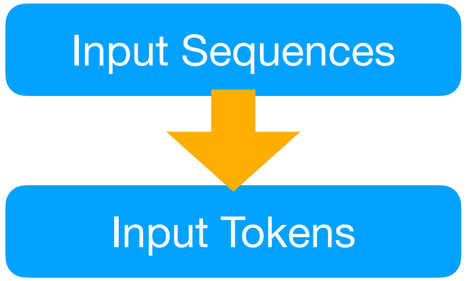
Based on GPT-4o

23 Characters
29 Tokens

学	48864
习	18259
的	254
意	9554
愿	37689
很	31374
重	123
要	17599
，	230
但	30358
汲	31634
取	3922
所	78388
学	21980
采	110
取	18655
行	32938
动	48864
才	13393
是	229
关	18655
键	23039
。	28833
	15355
	235
	21043
	30356
	62049
	1811

36 Characters
40 Tokens

배	39277
우려는	108
	41381
마음도	22289
	97
중요하지만	16969
	96677
,	49531
배운	49085
지식을	220
실제로	72043
실행에	36811
옮기는것이	88525
더	73653
중요합니다	11
.	74769
	94772
	67890
	77437
	18359
	30027
	97
	38187
	17835
	86888
	19954
	39623
	106
	21121
	16969
	28740
	225
	13094
	5251
	235
	242
	72043
	36811
	61938
	13



- Tokenization breaks down an input sequence into individual tokens.
- Based on the vocabulary of the model, each token is assigned a unique Token ID.
- Token IDs are numerical values, whereas tokens are still strings.
- The tokenization method is decided by model developers. It will also influence the vocabulary size.
- A token can a word, a sub-word, a character, an N-gram, or even a sentence.

Tokenizer	Year	Models
Byte-Pair Encoding (BPE)	2016	GPT-2,3,4, Phi3
SentencePiece	2018	ALBERT, T5, XLNet
Unigram	2018	T5
WordPiece	2019	BERT

Reference: Yang, Jinbiao. 'Rethinking Tokenization: Crafting Better Tokenizers for Large Language Models'. International Journal of Chinese Linguistics, vol. 11, no. 1, Jun. 2024, pp. 94–109. DOI.org (Crossref), https://doi.org/10.1075/ijchl.00023.yan.

Embeddings

*"**Embeddings** are numerical representations of real-world objects that machine learning (ML) and artificial intelligence (AI) systems use to understand complex knowledge domains like humans do."*

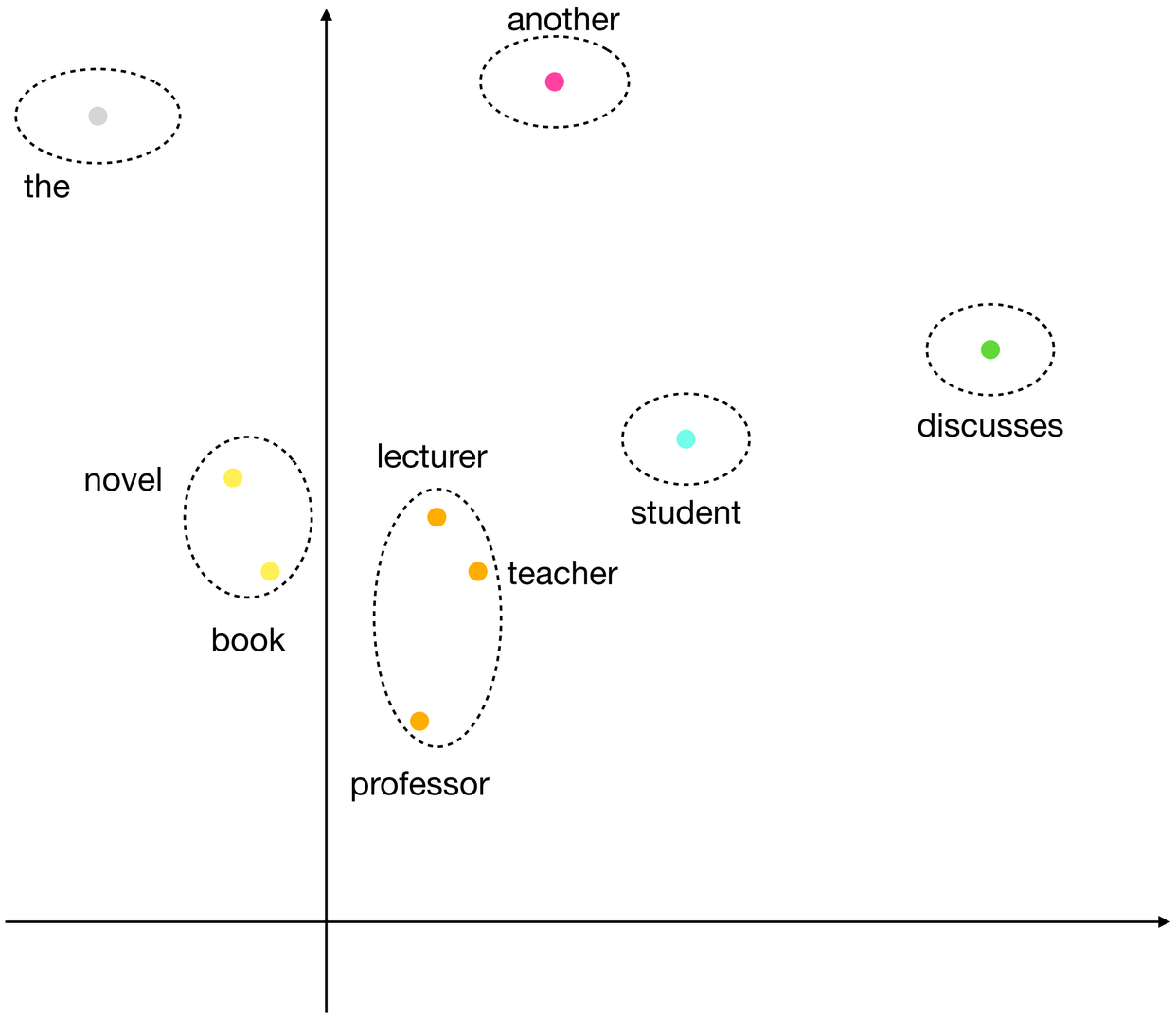
— Amazon.com

*"**Embedding** is a means of representing objects like text, images and audio as points in a continuous vector space where the locations of those points in space are semantically meaningful to machine learning (ML) algorithms."*

— IBM.com

- **Word Embeddings**
  - Each token is mapped to a high-dimensional space, represented by a series of numerical values (a vector).
  - Word embeddings can be easily generated from software libraries, such as Word2Vec, GloVe (Global Vectors for Word Representation), FastText, or BERT.
- **Positional Embeddings**
  - Add information about token position within a sentence to the word embeddings.
  - Note that transformer architecture does not explicitly capture position information of tokens.
- **Audio, Image, and Graph Embeddings**
  - These types of embeddings capture audio, visual features, vertex and link features, as well as semantic information about the content of these kinds.

Word Embedding Illustrated



Model	Dimension
BERT	768
GPT-2 (125M)	768
LLM2Vec-Llama-3	4096
DeepSeek-V3	7,168
GPT-3 (175B)	12,288

Example:

		Embedding	
Token	**Token ID**	**Dimension 1**	**Dimension 2**
“teacher”	1	0.189933	0.416156
“lecturer”	2	0.173392	0.422101
“professor”	3	0.170693	0.379916
“student”	4	0.392929	0.531111
“discusses”	5	0.679281	0.663918
“book”	6	-0.111203	0.492133
“novel”	7	-0.181203	0.503108
“the”	8	-0.293727	0.963233
“another”	9	0.232411	0.892734

Positional Embeddings

Token	Token ID	Word Embedding
the	8	[-0.293727, 0.963233]
book	6	[-0.111203, 0.492133]
discusses	5	[0.679281, 0.663918]
another	9	[0.232411, 0.892734]
book	6	[-0.111203, 0.492133]

The positions of the same token are not captured.

		Embedding	
Token	**Token ID**	Dimension 1	Dimension 2
“teacher”	1	0.189933	0.416156
“lecturer”	2	0.173392	0.422101
“professor”	3	0.170693	0.379916
“student”	4	0.392929	0.531111
“discusses”	5	0.679281	0.663918
“book”	6	-0.111203	0.492133
“novel”	7	-0.181203	0.503108
“the”	8	-0.293727	0.963233
“another”	9	0.232411	0.892734

- The order of the words in a sentence, paragraph, or document is very important for understanding its meaning in tasks like translation and text generation.
- Default design of LLM architectures is not capable of capturing positional information.
  - This applies to various types of data, including words, audio signal, and objects within an image.
- LLMs based on the Transformer architecture requires a mechanism for encoding positional information, e.g., models like GPT-x.

Prepare Embeddings for LLM Input

Token	Token ID	Token index
the	8	1
book	6	2
discusses	5	3
another	9	4
book	6	5

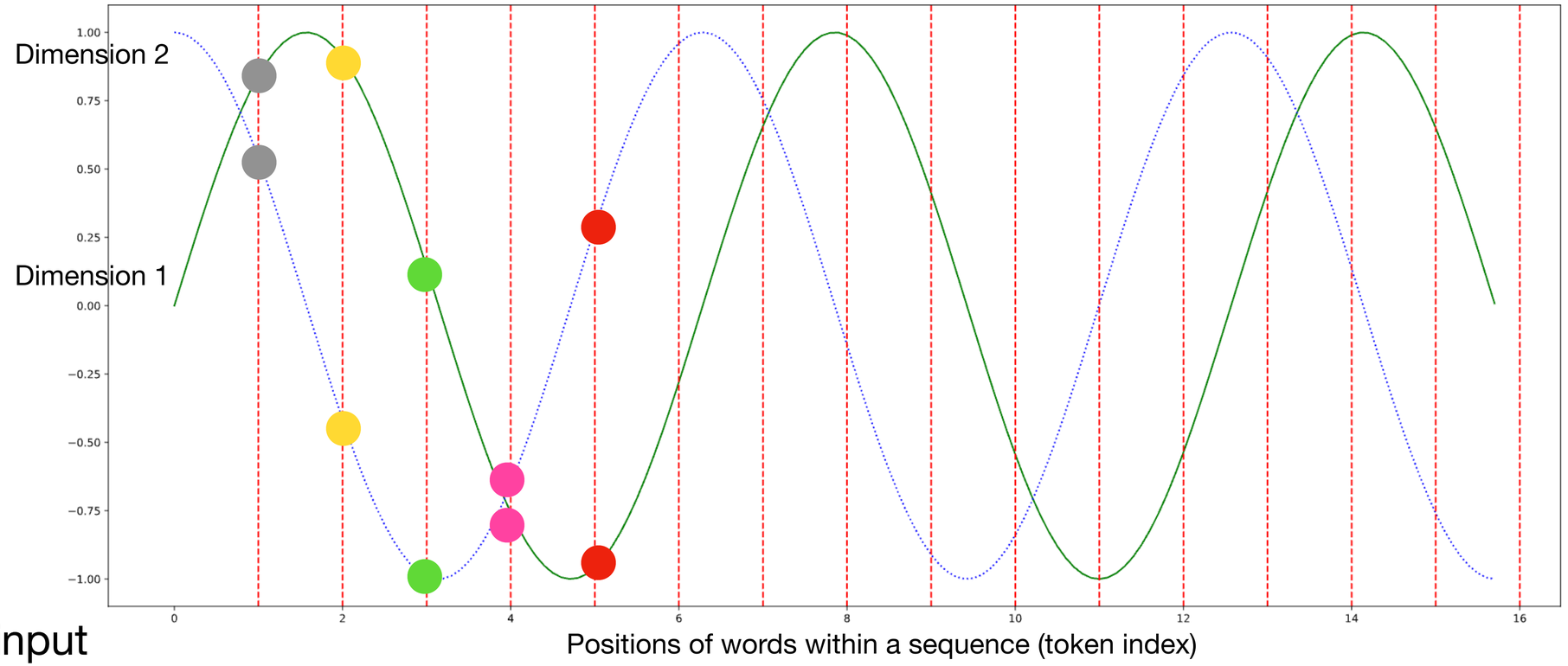
Word Embedding		Positional Embedding		Input to LLM
[-0.293727, 0.963233]	+	[0.841471, 0.540302]	=	[0.547744, 1.503534]
[-0.111203, 0.492133]		[0.909297, -0.416146]		[0.798094, 0.075987]
[0.679281, 0.663918]		[0.141120, -0.989992]		[0.820401, -0.326074]
[0.232411, 0.892734]		[-0.756802, -0.653643]		[-0.524391, 0.239091]
[-0.111203, 0.492133]		[-0.958924, 0.283662]		[-1.070127, 0.775795]

$$p_{i,2j} = sin\left(\frac{i}{10000^{\frac{2j}{dim}}}\right)$$

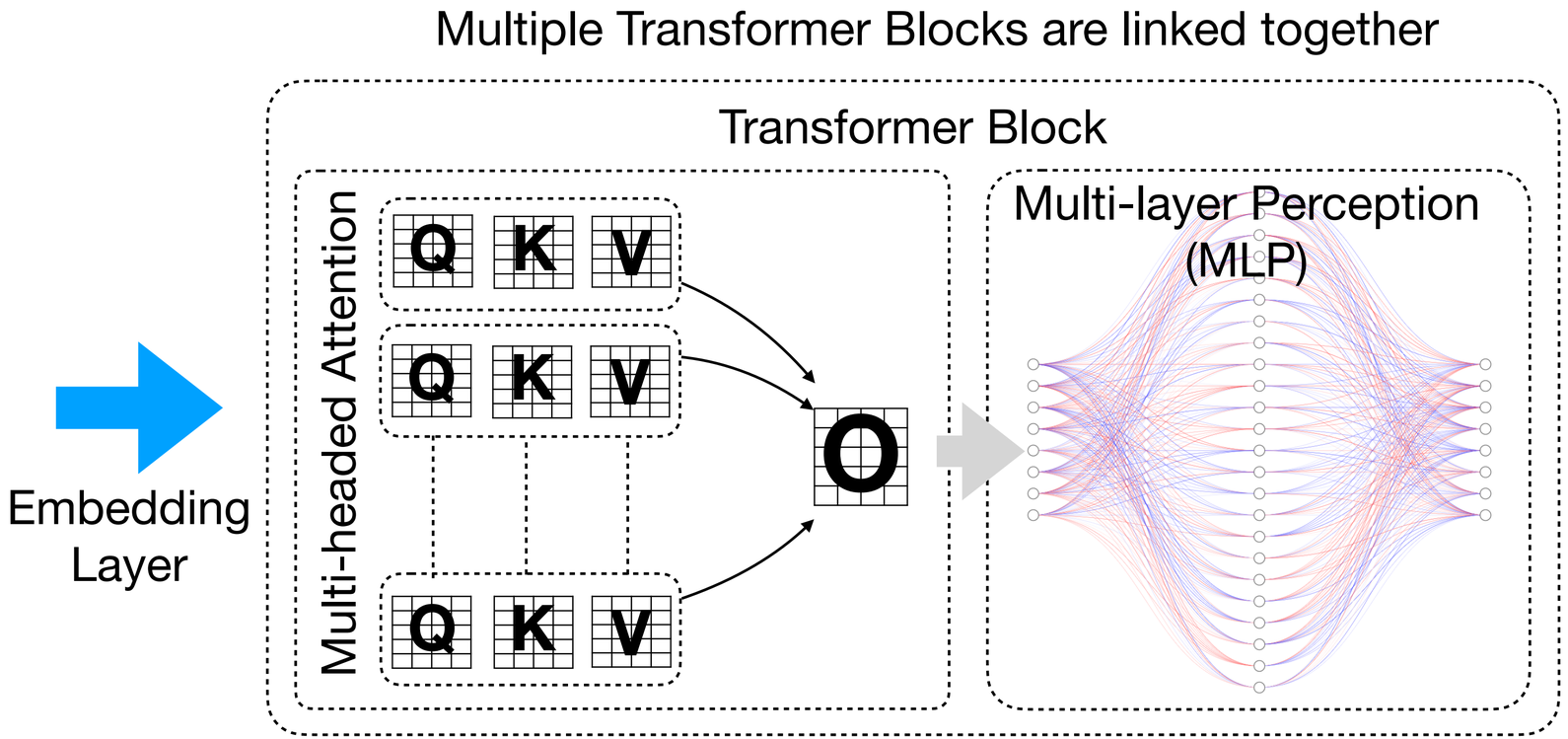
$$p_{i,2j+1} = cos\left(\frac{i}{10000^{\frac{2j}{dim}}}\right)$$

j : dimension index

i : position of the token in the input



Transformer Block (Simplified)

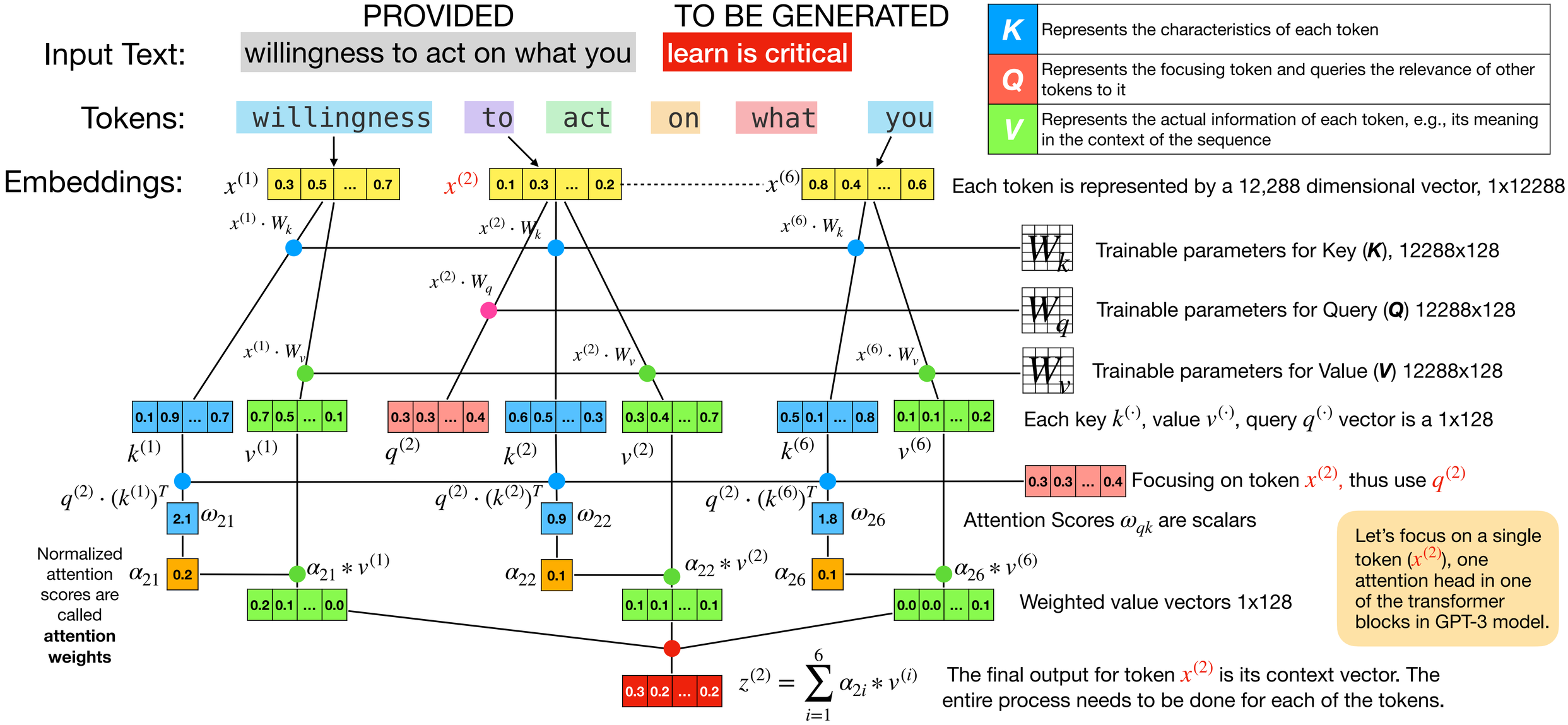


Model	Transformer blocks
GPT-2 Medium	24
GPT-2 Large	36
GPT-2 XL	48
GPT-3	96
Llama 2 7B	32
Llama 2 70B	80
Llama3 70B	80
Llama 3 405B	126

- Each transformer block typically comprises two key components: an attention module and a multi-layer perceptron (MLP) module.
  - **Attention Module**: consists of four weight matrices: Query, Key, Value and Output
  - **Multi-Layer Perceptron (MLP) Module**: consists of a series of linear layers, separated by nonlinear activation functions. Each linear layer is represented as a weight matrix that applies a linear transformation to the input information.
- It's worth noting that the **specific structure and contents of each transformer block can vary between different models and implementations.**

Self-Attention (1)

Each of the weight matrices (W_k, W_q, W_v) is 128 (row) * 12,288 (columns).
Recall that GPT-3 has 96 attention heads, thus each head has 12,288 / 96 =128 rows



K	Represents the characteristics of each token
Q	Represents the focusing token and queries the relevance of other tokens to it
V	Represents the actual information of each token, e.g., its meaning in the context of the sequence

Each token is represented by a 12,288 dimensional vector, 1x12288

W_k Trainable parameters for Key (***K***), 12288x128

W_q Trainable parameters for Query (***Q***) 12288x128

W_v Trainable parameters for Value (***V***) 12288x128

Each key $k^{(\cdot)}$, value $v^{(\cdot)}$, query $q^{(\cdot)}$ vector is a 1x128

0.3 0.3 ... 0.4 Focusing on token $x^{(2)}$, thus use $q^{(2)}$

Attention Scores ω_{qk} are scalars

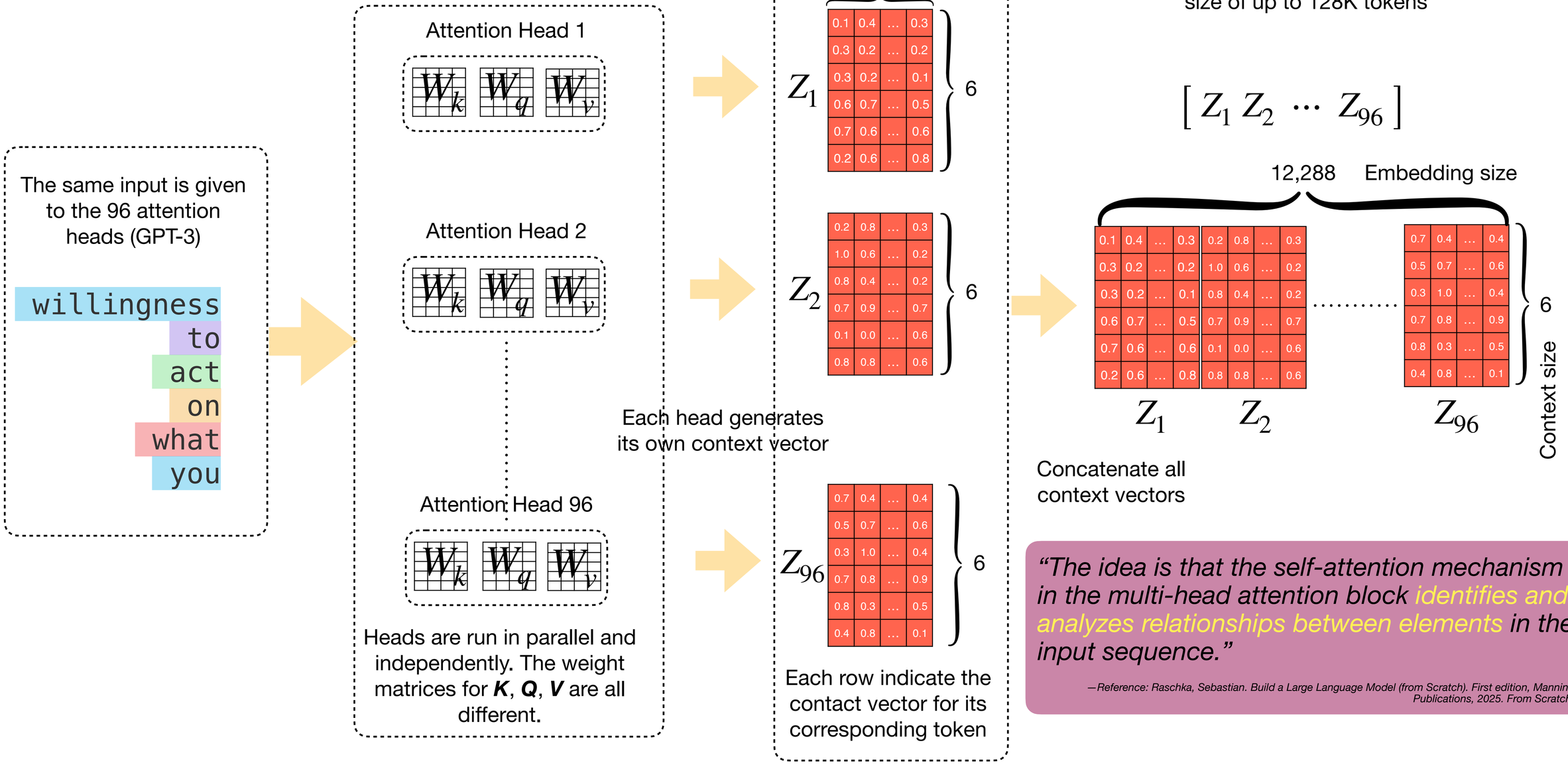
Weighted value vectors 1x128

Let's focus on a single token ($x^{(2)}$), one attention head in one of the transformer blocks in GPT-3 model.

The final output for token $x^{(2)}$ is its context vector. The entire process needs to be done for each of the tokens.

Reference: Raschka, Sebastian. Build a Large Language Model (from Scratch). First edition, Manning Publications, 2025. From Scratch.

Self-Attention (2)



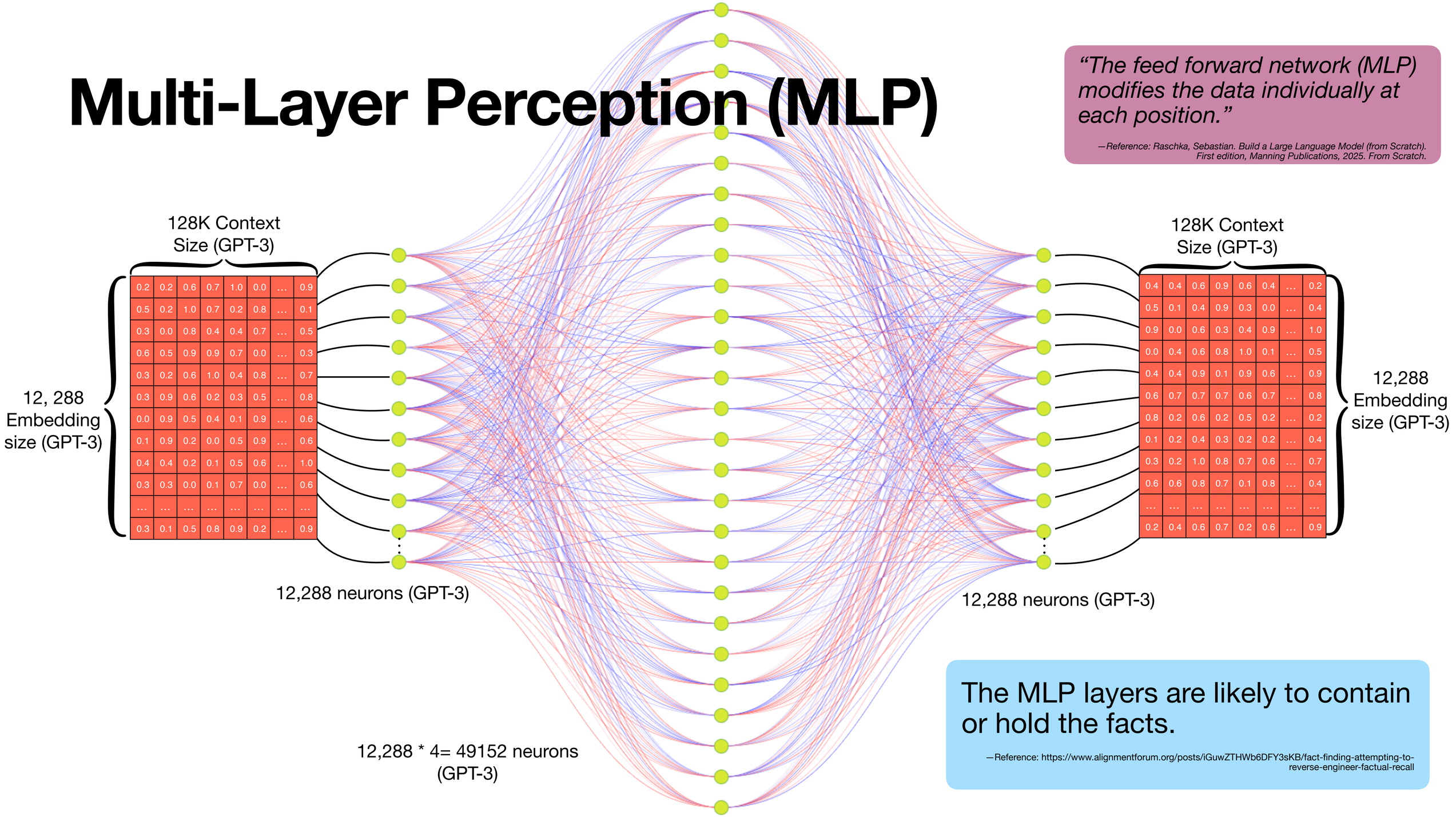
"The idea is that the self-attention mechanism in the multi-head attention block identifies and analyzes relationships between elements in the input sequence."

—*Reference: Raschka, Sebastian. Build a Large Language Model (from Scratch). First edition, Manning Publications, 2025. From Scratch.*

Multi-Layer Perception (MLP)

“The feed forward network (MLP) modifies the data individually at each position.”

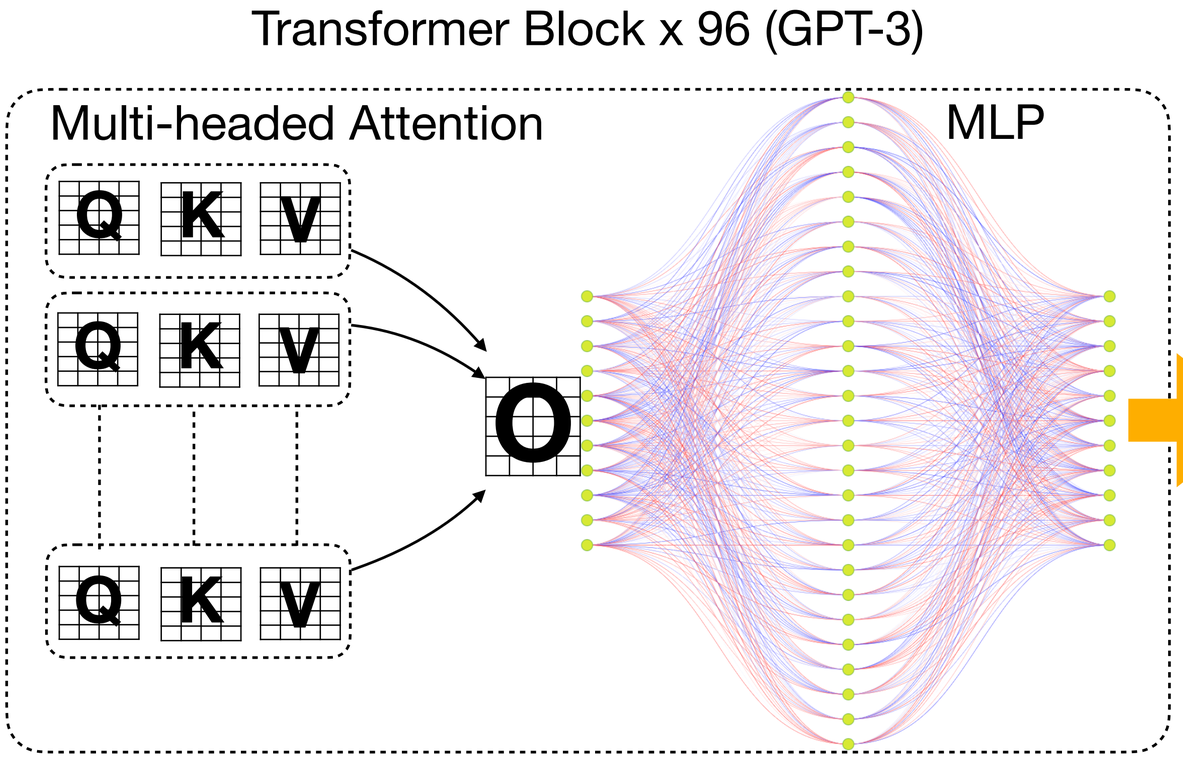
—Reference: Raschka, Sebastian. Build a Large Language Model (from Scratch). First edition, Manning Publications, 2025. From Scratch.

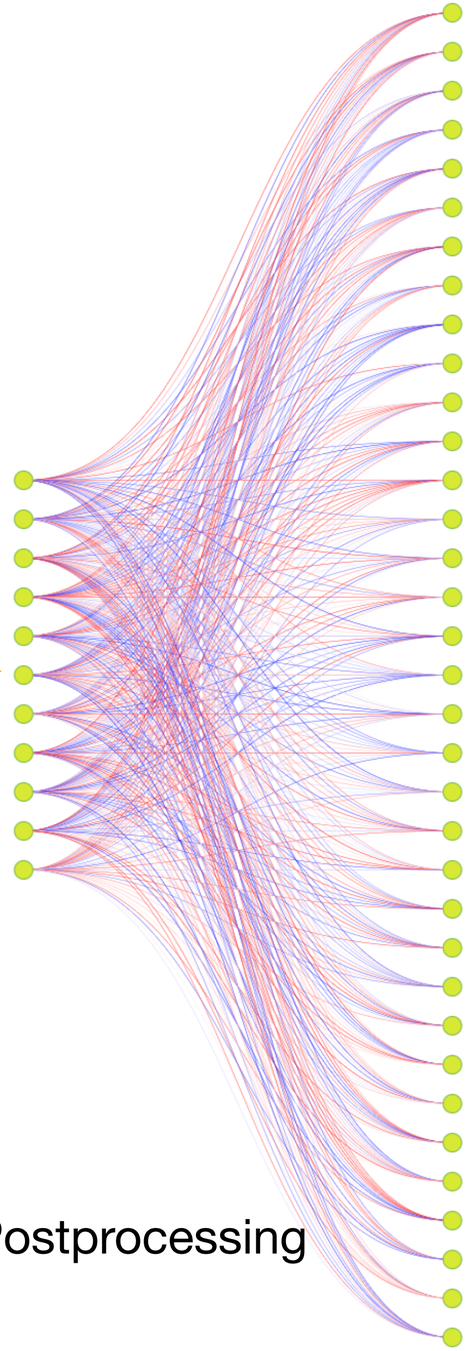


The MLP layers are likely to contain or hold the facts.

—Reference: https://www.alignmentforum.org/posts/iGuwZTHWb6DFY3sKB/fact-finding-attempting-to-reverse-engineer-factual-recall

Linear Output Layer





- GPT-2 and GPT-3 trained using the BPE (Byte-Pair Encoding) tokenizer have a vocabulary size of 50,257.
- Each output corresponds to the likelihood of the word being selected.

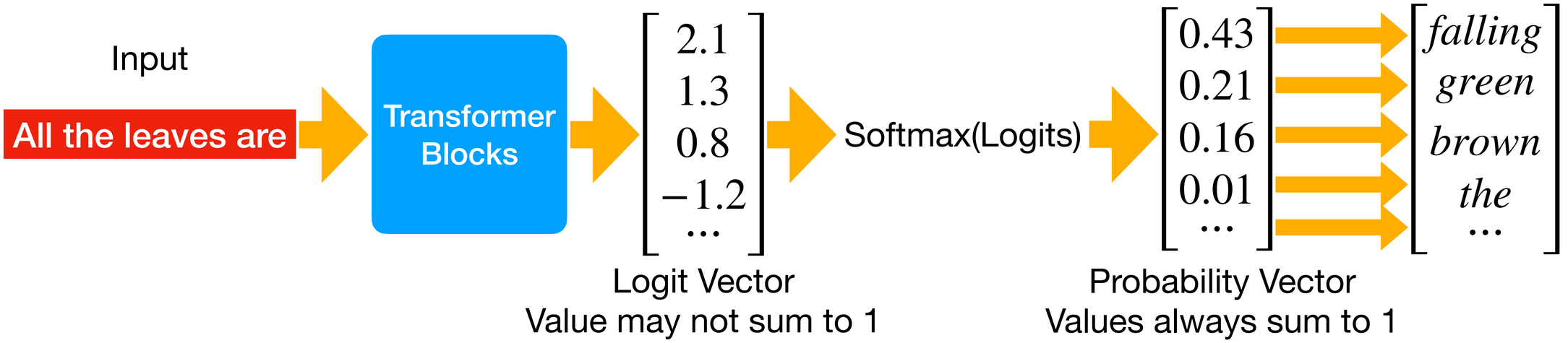
Postprocessing

Unembedding

Sampling

$$p_i = \text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

- Given an input, the network generates a *logit* vector, each logit corresponds to one token in the model's vocabulary.
- The logits are not probabilities; they don't sum to 1, because a logit represents a relative score between tokens.
- A common practice is to use a '*softmax*' function to convert logits into probabilities.

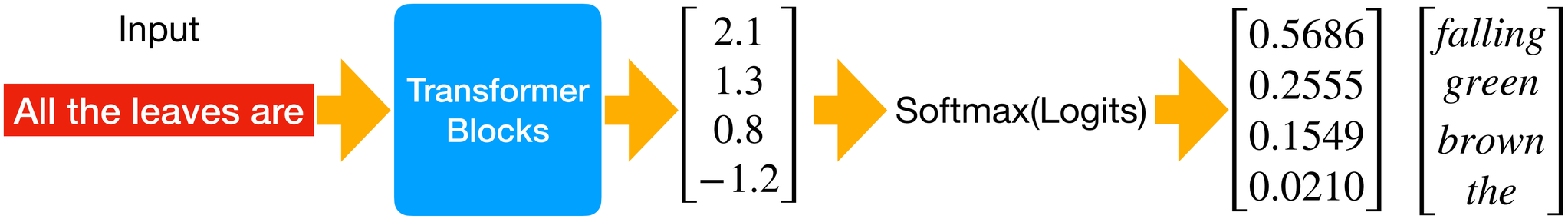


*"**Logits** are the raw output values generated by a model before any probability transformation is applied. They represent the model's unnormalized predictions for every possible token at each step of text generation."*

— https://docs.lm-kit.com/lm-kit-net/guides/glossary/logits.html

Sampling with *Temperature*

- If we always choose the highest probability token, the results will always be the same for the input, i.e., no creativity.
- Temperature is a constant that adjusts the logits before the softmax transformation.
- Example, assume our vocabulary only contains four words:



Temperature (T) = 0.2

$\begin{bmatrix} 2.1/T \\ 1.3/T \\ 0.8/T \\ -1.2/T \end{bmatrix}$ Softmax(Logits) $\begin{bmatrix} 0.9806 \\ 0.0179 \\ 0.0015 \\ 0.0000 \end{bmatrix}$

Temperature (T) = 1.5

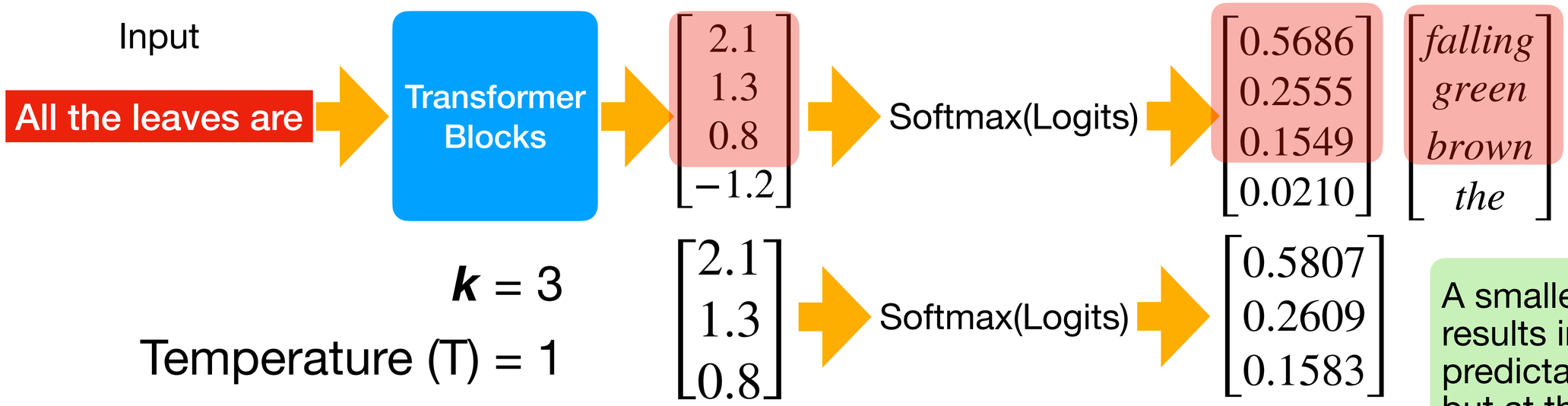
$\begin{bmatrix} 2.1/T \\ 1.3/T \\ 0.8/T \\ -1.2/T \end{bmatrix}$ Softmax(Logits) $\begin{bmatrix} 0.4722 \\ 0.2770 \\ 0.1985 \\ 0.0523 \end{bmatrix}$

With a higher temperature, tokens with lower probabilities have a greater likelihood to be selected.

Sampling with *Top-k*

$$p_i = \text{softmax}(x_i) = \frac{e^{x_i}}{\sum_j e^{x_j}}$$

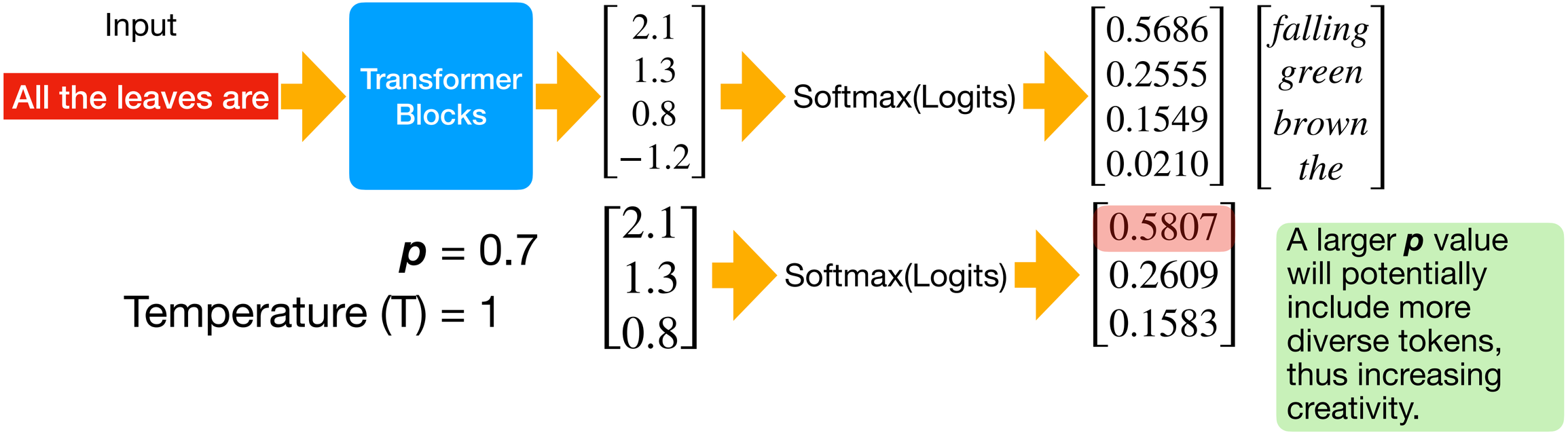
- Softmax calculation can be computationally intensive, especially when the model's vocabulary is large, for example, Llama 3 has a vocabulary size of 128,000.
- To reduce the computation, we can select the top ***k*** tokens from the logit vector and then use the softmax to convert the top-*k* logits into probabilities.
- A typical value for *k* in practice can be range from 50 to 500.
- Using the same example:



A smaller ***k*** value results in more predictable text, but at the cost of reduced creativity.

Sampling with *Top-p*

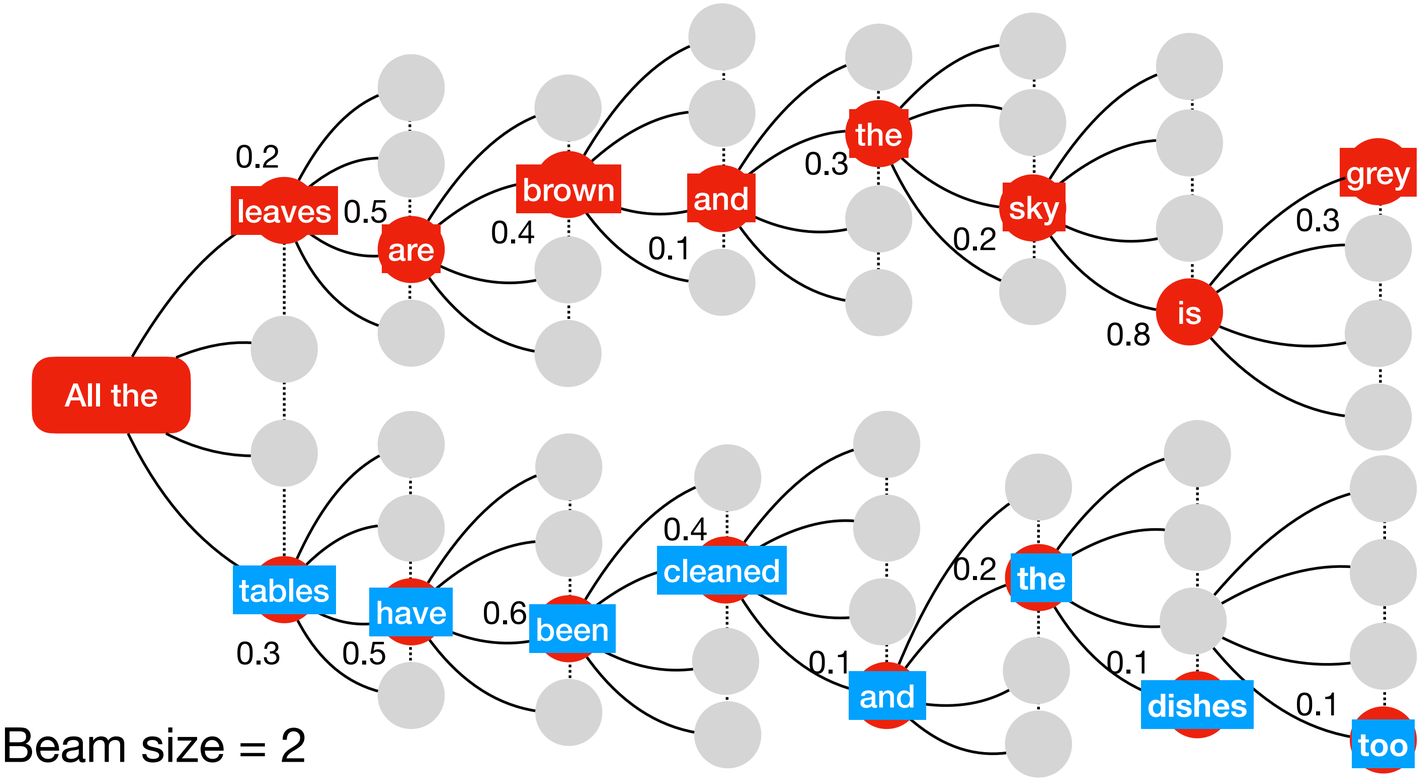
- In top-*p* sampling, the model sums the probabilities of the most likely tokens until it reaches a specific value, ***p***.
- Top-p sampling is also known as nucleus sampling, typically range from 0.9 to 0.95.



$$\frac{1}{L^{\alpha}} \log P\left(y_1, y_2, \cdots, y_L \mid \mathbf{c}\right) = \frac{1}{L^{\alpha}} \sum_{t'=1}^{L} \log P\left(y_{t'} \mid y_1, \cdots, y_{t'-1}, \mathbf{c}\right)$$

Sampling with Beam Search

- Instead of generating a single response per query, generating multiple responses can increase the likelihood of good responses.
- Beam search offers a trade-off between accuracy and computational cost through the flexible selection of the beam size.



Beam size = 2

All the leaves are brown and the sky is grey

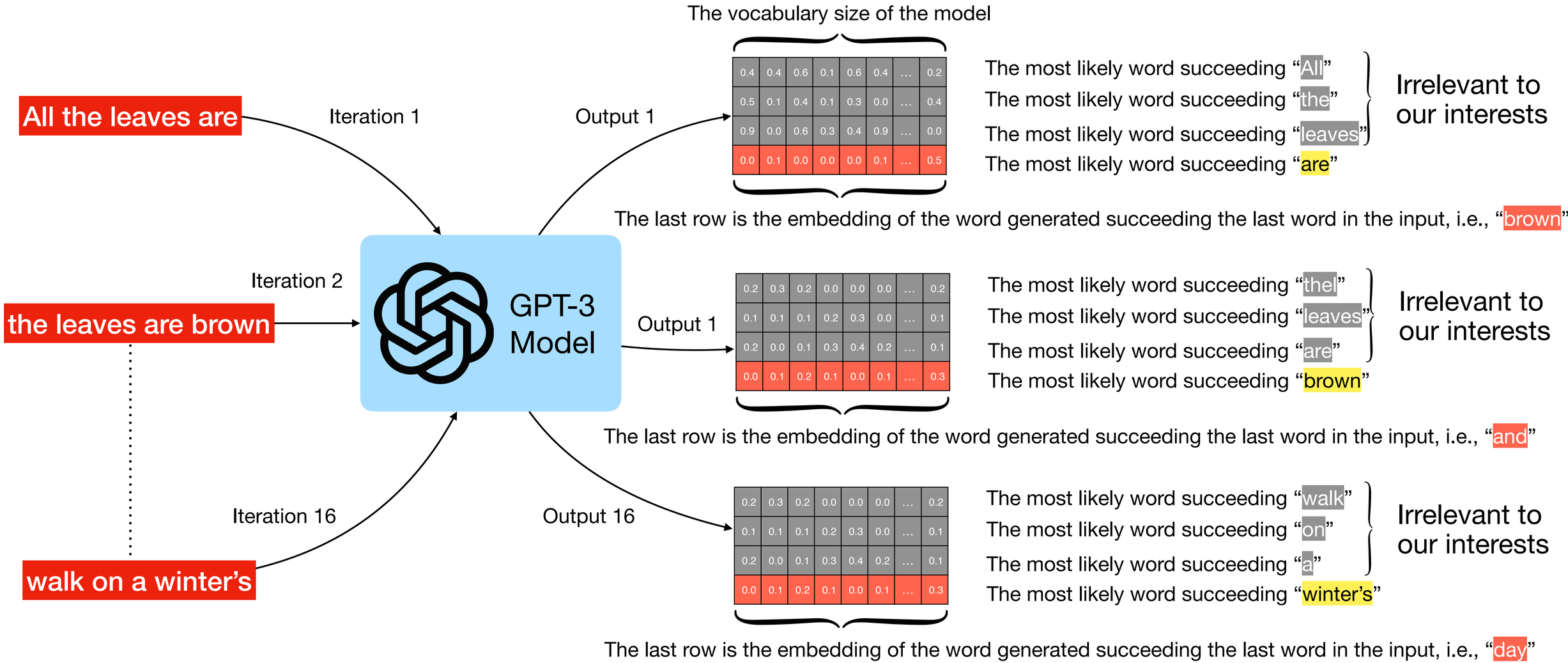
p1 = 0. 2 * 0.5 * 0.4 * 0.1 * 0.3 * 0.2 * 0.8 * 0.3 =0.0000576

p1 > p2

p2 = 0.3 * 0.5 * 0.6 * 0.4 * 0.1 * 0.2 * 0.1 * 0.1 =0.0000072

All the tables have been cleaned and the dishes too

Putting Everything Together



Completed Text: All the leaves are brown and the sky is grey I have been for a walk on a winter's day

Size of GPT-3 Example

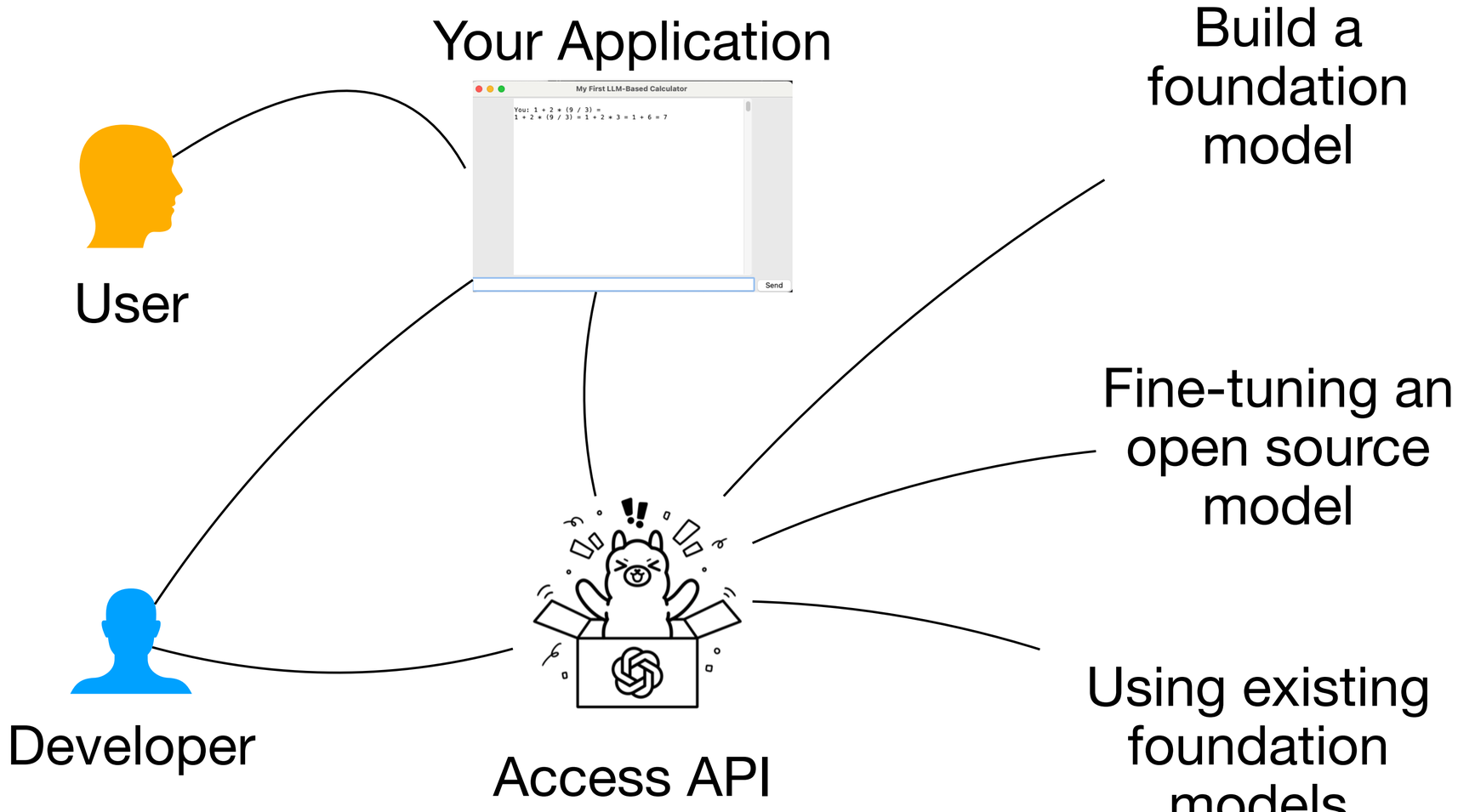
Word Embedding Dimension	Vocabulary Size	Number of Attention Heads	Number of Transformer Blocks
12,288	50,257	96	96

Input	**Embedding**	dim-embedding * size-of-vocabulary 12,288 * 50,257	617,558,016	1 matrix
Attention Layer	**Key**	dim-query * dim-embedding * num-of-head * num-of-blocks 128 * 12,288 * 96 * 96	14,495,514,624	Each attention head has its own weight matrix, thus 96 heads * 96 blocks = 9216 matrices
	Query	dim-query * dim-embedding * num-of-head * num-of-blocks 128 * 12,288 * 96 * 96	14,495,514,624	Each attention head has its own weight matrix, thus 96 heads * 96 blocks = 9216 matrices
	Value	dim-value * dim-embedding * num-of-head * num-of-blocks 128 * 12,288 * 96 * 96	14,495,514,624	Each attention head has its own weight matrix, thus 96 heads * 96 blocks = 9216 matrices
	Output	dim-value * dim-embedding * num-of-head * num-of-blocks 128 * 12,288 * 96 * 96	14,495,514,624	At each block, multi-head attentions merge to one output, thus: 96 blocks = 96 matrices
MLP Layer	**Projection Layer 1**	Each layer contains 4 * 128,882 neutrons, thus num-of-neurons * dim-embedding * num-of-blocks	57,982,058,496	Each layer contains one matrix, thus 1 * 96 blocks = 96 matrices
	Projection Layer 2	Each layer contains 4 * 128,882 neutrons, thus num-of-neurons * dim-embedding * num-of-blocks	57,982,058,496	Each layer contains one matrix, thus 1 * 96 blocks = 96 matrices
Output	**Unembedding**	size-of-vocabulary * dim-embedding 50,257 * 12,288	617,558,016	1 matrix
		Total Weights:	**175,181,291,520**	**27,938 matrices**

Inside the World’s Largest AI Supercluster xAI Colossus



Using LLMs in Your Applications



High cost & high technical barriers

Low cost & low technical barriers

Summary

- **LLMs** are very large deep learning models that have been pre-trained on vast amounts of data.
- Depending on their intended use, LLMs can be categorized as **Masked Language Models (Encoders)** or **Autoregressive Language Models (Decoders)**.
- Pre-trained models are often referred to as **foundation models**.
- For LLMs to perform well in domain-specific tasks, they need to be **fine-tuned** on relevant datasets.
- The **length of input prompts** is limited; long inputs must be split into smaller chunks for processing.
- **Embeddings** are numerical representations of real-world objects that machine learning and AI systems use to understand complex knowledge domains.
- The **self-attention mechanism** in the multi-headed attention block identifies and analyzes relationships between elements in the input sequence.
- In transformer architectures, the **multi-layer perceptron (MLP)** network modifies data individually at each position; this might be a place where facts are stored.
- The **sampling techniques** and relevant parameter tuning define the creativity of the model.